



PYTHON 101 // GROUP PRESENTATION

Python Fundamentals

Variables, Data Types & Strings

A walkthrough of core Python building blocks — naming variables, working with data types, converting between them, manipulating strings, and formatting output with f-strings.



```
name = "Python"  
version = 3.12  
print(f"{name} {version}")
```

Presented by: _____



TEAM

Who Did What



Simran

PART 1

Variables & Data Types

Naming rules, best practices, and Python's core data types



Md Sahil Manzoor

PART 2

Type Casting & String Methods

Conversion functions and built-in string operations



Prattay Bari

PART 3

f-Strings & Slide Design

String formatting, visuals, and presentation assembly

What We'll Cover



Variables

Naming rules & best practices

1



Data Types

int, float, str, bool, NoneType

2



Type Casting

int(), str(), float(), bool()

3



String Methods

upper(), lower(), len(), strip()

4



f-Strings

Modern string formatting

5

```
# topics.py
topics = [
    'vars',
    'types',
    'casting',
    'strings',
    'f-str'
]
```

Naming Rules & Best Practices



Naming Rules

- Must start with a letter or underscore (_), never a digit
- Can contain letters, digits and underscores only
- Names are case-sensitive: age and Age differ
- Cannot use Python reserved keywords (e.g. class, for, if)
- No spaces or special characters (-, @, %, etc.)



Best Practices

- Use descriptive, meaningful names: total_price not tp
- Follow snake_case for variables and functions
- Use UPPER_CASE for constants: MAX_SPEED = 120
- Avoid single letters except for short-lived loop counters



```
user_age = 21           # ok   clear
MAX_RETRIES = 5         # ok   constant
2nd_value = 10          # x    starts w/ digit
class = "A"             # x    reserved word
```

Python's Core Data Types



int

Whole numbers, positive or negative

```
age = 25
```



float

Decimal / floating-point numbers

```
price = 9.99
```



str

Text, sequences of characters

```
name = "Sam"
```



bool

Logical True / False values

```
is_on = True
```



NoneType

Represents the absence of a value

```
data = None
```

Use Python's built-in `type()` function to check a variable's data type at any time — e.g. `type(age)` returns `<class 'int'>`.

Converting Between Data Types

Type casting converts a value from one data type to another using built-in functions. This is essential when combining inputs, doing math on strings, or formatting output.

**int()**

Converts a string or float to an integer

```
int("42") → 42
```

**str()**

Converts any value into its text form

```
str(42) → "42"
```

**float()**

Converts a string or int to a decimal

```
float("3.5") → 3.5
```

**bool()**

Converts a value to True or False

```
bool(0) → False
```

Watch out: `int("3.5")` raises a `ValueError` — convert to float first, then to int if needed.

Common String Methods

**.upper()**

Returns the string in all uppercase letters

**.lower()**

Returns the string in all lowercase letters

**len()**

Returns the number of characters in a string

**.strip()**

Removes leading/trailing whitespace



```
s = "  Hi There  "
```

```
s.upper()
```

```
# 'HI THERE'
```

```
s.lower()
```

```
# 'hi there'
```

```
len(s)
```

```
# 13
```

```
s.strip()
```

```
# 'Hi There'
```

f-Strings & String Formatting

f-Strings (formatted string literals) let you embed expressions directly inside a string using curly braces — introduced in Python 3.6 as the modern, readable way to format text.

Old way (.format)

```
"Hi {}, you are {}".format(name, age)
```

Modern way (f-string)

```
f"Hi {name}, you are {age}"
```

- Prefix the string with f or F before the quotes
- Place variables or expressions inside { }
- Supports inline calculations: f"{a + b}"
- Format numbers with specifiers: f"{pi:.2f}"

```
name = "Maya"
age = 21
pi = 3.14159

print(f"Hi {name}!")
# Hi Maya!

print(f"Age: {age}")
# Age: 21

print(f"Pi: {pi:.2f}")
# Pi: 3.14
```




CONCLUSION

Key Takeaways



Clear, rule-following variable names make code readable and maintainable



Python's five core types — int, float, str, bool, NoneType — cover most everyday data



Type casting functions let values move safely between these types



Built-in string methods handle common text clean-up and transformation



f-Strings are the fastest, most readable way to format output in modern Python



```
name = "Team"
topics = 5
done = True

print(f"Thanks, {name}!")
# Thanks, Team!
```

Thank you — Questions Welcome